



```
def deploy_pipeline():
    project = "AI"
    env = "prod"
    → deploy_result =
        agent = CodingAgent(model="gpt-4o")
        await agent.analyze(project)
        tests = await agent.run_tests()
        if tests.passed:
            return await agent.deploy(env)

# Execução da Pipeline
await pipeline = run_AIPipeline(
    model="Claude-opus",
    tests=["test", "testenv", "testprod"],
    max_iterations=50
)

# Resultado da Pipeline
print(await pipeline.execute())
# Saída: Sistema completo
# de gestão de pedidos com
# testes unitários e deploy
```

— DESENVOLVIMENTO DE SOFTWARE —

Análise da Fábrica Agêntica

Arquitetura Limpa | Código Limpo | Segurança | UI/UX

◆ NÍVEL TÉCNICO

16 capítulos

projetos práticos

• • •

AUTOR

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

16
CAPÍTULOS

70+
PÁGINAS

∞
PROJETOS

Heverton Eduardo Peres

Análise da Fábrica Agêntica

Arquitetura Limpa | Código Limpo | Segurança | UI/UX

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

RELATÓRIO DE ANÁLISE — FÁBRICA AGÊNTICA DE PUBLICAÇÕES	4
1. ANÁLISE DE ARQUITETURA LIMPA	4
1.1 Visão geral	4
1.2 Pontos fortes	5
1.3 Pontos de atenção / degenerações sintomáticas	7
2. ANÁLISE DE CÓDIGO LIMPO	8
2.1 Legibilidade	8
2.2 DRY / Coesão / Acoplamento	8
2.3 Testabilidade	9
2.4 Manejo de erros e limites	9
3. FALHAS E BUGS DE SEGURANÇA	9
3.1 Não mitigado / de design	9
3.2 Mitigado (bom trabalho documentado)	11
4. ANÁLISE DE UI/UX	11
4.1 Interação operador ↔ fábrica	11
4.2 UI/UX dos entregáveis gerados	12
4.3 Máquina de vendas	12
4.4 Pontos de fricção de UX	13
5. SÍNTESE	14
6. O QUE NÃO ENTRA NESTE RELATÓRIO (por escopo)	16

RELATÓRIO DE ANÁLISE — FÁBRICA AGÊNTICA DE PUBLICAÇÕES

Projeto: proj_fabrica-de-livros

Versão analisada: V5 (coleção multi-formato) + evolução histórica V3→V5.5

Data: 18/08/2026

Analísador: Solar Pro 4 (via Hermes Agent)

Escopo: análise de arquitetura, código, segurança e UI/UX — opinião técnica sem alteração de artefatos

1. ANÁLISE DE ARQUITETURA LIMPA

1.1 Visão geral

A fábrica é um **sistema orquestrador de produção de conteúdo editorial** com tipificação declarativa (8 tipos de obra: Livro, TCC, Artigo, E-book, Playbook, Lead Magnet, Deck, E-mails), pipeline determinístico (scripts Python) + agentes LLM, geração de múltiplos formatos (PDF, EPUB, HTML, PPTX), camadas opcionais de campanha e máquina de vendas, e portabilidade multi-IDE via hardlinks/junctions.

1.2 Pontos fortes

Aspecto	Observação
Registro declarativo central	<code>scripts/tipos_obra.py</code> é o único lugar para adicionar um tipo novo — os 6 pontos de dispatch consultam o registro. Isso é decisão de design limpa: editar 1 arquivo, não 6.
Separação de responsabilidades por fase	Fase 1 (P&D) → Fase 2 (manufatura paralela) → Fase 2.5 (peer review) → Fase 3 (acabamento) → Fase 4 (coleção derivada). Cada fase tem agentes/skill próprios e contratos de dados explícitos (JSON de estado, pool, auditoria).
Deterministicismo onde importa	Scripts como <code>indexar-dossie.py</code> , <code>pool-capitulos.py</code> , <code>validar-codigo.py</code> , <code>auditar-obra.py</code> , <code>renderizar-diagramas.py</code> são determinísticos. Os gates de conteúdo (R-RF, R-MT, R-ES, R-AF, R-FT) adicionam verificação de mérito além da estrutura.
Desenho multi-IDE sólido	Hardlinks (<code>AGENTS.md</code> ↔ <code>CLAUDE.md</code>), junctions (<code>agentic/*</code> , <code>.agents/*</code> , <code>.opencode/*</code> → <code>.claude/*</code>), scripts de sincronização de MCP (<code>sync-vscode-mcp.mjs</code> , <code>sync-opencode-mcp.mjs</code>). O <code>.git/hooks/pre-commit</code> é copiado, não linkado — correto, pois <code>.git/hooks</code> não aceita junction confiável.
Economia de tokens como princípio de arquitetura	Regras 0.1–0.12 documentadas e aplicadas (caveman, headroom, lean-ctx, delegação cavewcrew). Pesquisa acadêmica V5.4 é custo-zero via APIs abertas. Artigos/ebooks reaproveitam dossiê indexado — não pesquisam do zero.
Estação resistente a falhas	Backoff exponencial por capítulo (máx. 3), pool registra falha e marca <code>esgotado</code> , Pandoc/Typst ausentes → Markdown expedido e pendência reportada (não bloqueia), mermaid-cli ausente → diagrama permanece como código no PDF.

1.3 Pontos de atenção / degenerações sintomáticas

1. Caminhos hardcoded de binaries (Windows-only)

- `compilar-para-pdf.py:97-98` fixa os paths do Pandoc e Typst derivados de uma instalação específica via WinGet. Isso quebra portabilidade (outro usuário/URL WinGet diferente) e impede execução em Linux/macOS sem edição. A arquitetura multi-IDE e multi-plataforma prometida no AGENTS.md §6 não se estende ao pipeline de compilação. O ideal: resolver via `shutil.which` com fallback para paths configuráveis, ou variável de ambiente.

2. Hardcodificação do autor no template

- `compilar-para-pdf.py:336`: `"-V", "author=Heverton Eduardo Peres"` — fixo no código, não vem de `config_obra.json` nem de metadados. Para uma fábrica que produz obras de múltiplos autores, isso é um vazamento de responsabilidade: o autor deveria ser campo do registro de obra.

3. Imports relativos via `sys.path.insert`

- `compilar-para-pdf.py:26`:
`sys.path.insert(0, str(Path(__file__).parent / "scripts"))` — funciona, mas é menos limpo que um pacote Python instalável ou `PYTHONPATH`. Se o projeto crescer ou ganhar test coverage no script, isso fica frágil.

4. Dualidade plana vs. hub por coleção ainda deixa rastros

- O AGENTS.md acrescentou um bloco RTK (l. 458-462) registrando que `fatiar-obra.py --playbook` gravou em `output/playbooks/` (fora do hub) e `minerar-fontes-academicas.py` gravou em `output/<slug>/pesquisa/` (fora do hub). Isso é um sintoma de que a regra “SEMPRE usar `tipos_obra.dir_obra(slug)`” não é verificada antes da gravação — depende da disciplina do agente. Um guardião de caminho no `tipos_obra.py` (ex.: `_assert_dentro_do_hub`) seria mais robusto que confiar na memória do agente.

5. Subagente de expansão travou em 48 turns (l. 440-444)

- Registrado no RTK: spawnar 1 subagente para expandir 8 capítulos estoura o limite de turns. Isso não é falha de arquitetura, mas é evidência de que a granularidade da tarefa não foi calibrada. A prevenção sugerida (lotes de 2-3) é boa, mas é uma prevenção documental, não estrutural.

6. Código de configuração muito disperso entre scripts e JSONs

- `config_obra.json`, `tipos_obra.py`, `metadados_livro.py`, `parametros_obra.py`, `series_capa.py`, `colecacao.py`, `campanha.py`, `scripts/hooks/pre-commit`, `puppeteer.config.cjs`, `opencode.json`, `.mcp.json`, templates Typst, templates HTML de campanha — todos carregam pedaços da verdade. Não há um `config_schema` único ou um `json-schema` do contrato de estado. Quando surge um bug de derivação (ex.: `obra_mae` vs. `serie/livro_mae`, l. 243), a raiz é que vários scripts leem campos com nomes diferentes e só a experiência mostra qual é o canônico.

2. ANÁLISE DE CÓDIGO LIMPO

2.1 Legibilidade

Positivos: - Docstrings em português no início dos scripts explicando propósito, pipeline e modo de uso. - Funções com nomes declarativos: `converter_via_typst`, `variaveis_visuais`, `comando_pandoc`, `renderizar_diagramas`, `validar_playbook`. - Código de validação de código (`validar-codigo.py`) é bem estruturado: dicionário `VALIDADORES`, regex `RE_BLOCO`, separação clara entre validação estática e execução (`--executar`).

Atenções: - `compilar-para-pdf.py` tem ~30 imports de `SLUGS_*` via `try/except` (l. 41-92). Isso esconde a configuração real em 8 módulos opcionais que podem ou não existir — dificulta saber quais livros serão compilados sem rastrear 8 arquivos. Uma lista única ou carregamento via JSON/manifest seria mais legível. - `compilar-para-pdf.py:524-531`: frontmatter YAML montado com string f. Ok para o tamanho, mas se o autor/data/linguagem virarem campos do registro, esse trecho deve sair para `metadados_livro`. - `validar-codigo.py:229-238`: `detectar_linguagem` é heurística com regex — aceitável para `playbook cards` sem tag, mas pode misclassificar. Não há teste visível para essa função.

2.2 DRY / Coesão / Acoplamento

O que está bom: - `tiposobra.py` como registro central remove duplicação de `dispatch` por tipo. - `validar-codigo.py` tem uma única função `validar_arquivo` e `validar_playbook` que compartilham a lógica de extração de blocos e aplicação de validadores. - `compilar-para-pdf.py` delega a `tiposobra.compilador_de` para tipos com motor próprio — evita `if/elif` por tipo.

O que sente-se repetitivo ou acoplado: - Tratamento de imports opcionais (`try: import X except ImportError: X=None`) é repetido em `compilar-para-pdf.py` (`metadados_livro`, `parametrosobra`, `tiposobra`) e provavelmente em outros scripts. Uma função util `import_opcional(name)` ou um módulo `fabrica_imports` reduziria boilerplate. - Múltiplos scripts leem `configobra.json` de forma independente — se o schema mudar, há N lugares para atualizar. O `parametrosobra.py` parece ser o canônico, mas não é claro que todos os scripts passam por ele. - Template Typst e template HTML são mantidos separados — OK, são mundos diferentes (PDF vs. Chromium). Porém, a lógica de **gable**, **paleta**, **ficha catalográfica** aparece em `metadados_livro.py` (Python) e nos templates Typst — se a paleta mudar, há dois lugares.

2.3 Testabilidade

- `pytest.ini` existe + `tests/` com testes (provavelmente `test_auditar_obra.py`, `test_parametros_obra_v5.py`, `test_campanha.py`, `test_colecao_hub.py`, `test_maquina_colecao.py`, conforme registrado no RTK).
- `validar-codigo.py` é testável por natureza — função pura que recebe texto e devolve lista de registros.
- R16 (nunca commitar vermelho) é uma regra de processo sólida, e o hook `pre-commit` que bloqueia commit se `pytest -q` falhar é a mecanização correta.

Atenção: - O script de compilação (`compilar-para-pdf.py`) depende de Pandoc e Typst instalados em paths específicos — testes que dependem desses binaries são frágeis em qualquer máquina que não tenha a configuração exata. O RTK menciona `precisa_pandoc_typst` como skip em testes — correto, mas indica que a cobertura do caminho de compilação real é limitada.

2.4 Manejo de erros e limites

- `compilar-para-pdf.py` tem `try/except` bem colocado ao redor do `subprocess.run` com `TimeoutExpired` e `Exception` genérico, e reporta o erro com stdout/stderr truncado. Bom.
- `validar-codigo.py` tem `_rodar` com `FileNotFoundError`, `TimeoutExpired`, `Exception` — cobertura adequada.
- Um ponto: `subprocess.run` com `capture_output=True` e timeout é usado em vários lugares, mas em `validar-codigo.py:269-271` o `cwd=td` (tempdir) está correto para sandbox leve. Para blocos bash arbitrários, um tempdir não é sandbox completo (o código pode ler arquivos absolutos) — mas para o uso pretendido (smoke test de código de capítulo/playbook) é aceitável.

3. FALHAS E BUGS DE SEGURANÇA

Nota: “segurança” aqui é avaliada no contexto de **sistema de produção de conteúdo editorial orquestrado por agentes**, não como aplicação web voltada a usuário final (embora a máquina de vendas seja um caso parcial).

3.1 Não mitigado / de design

1. Execução de código arbitrário via `--executar`

- `validar-codigo.py:8-10`: “Com `--executar`, os blocos executáveis (python/javascript/bash) são também EXECUTADOS em sandbox leve”.
 - `executar_bloco` (l. 241-283): executa python com `sys.executable -c`, node com `node -e`, bash com `bash -c`, em tempdir com env minimalista.
 - **Risco**: qualquer agente/redator que injete um bloco de código malicioso (ou acidentalmente destrutivo) em um capítulo/playbook e passe `--executar` está a executar. O tempdir restringe gravações, mas o código pode ainda assim fazer leitura de arquivos sensíveis via caminhos absolutos, ou emitir rede se houver acesso. Para um projeto solo/perfeitamente confiável é baixo risco; para um projeto com múltiplos colaboradores ou com prompts genéricos de LLM, o risco sobe.
 - **Mitigação recomendada**: deixar `--executar` explícito e documentado como “apenas para conteúdo de confiança”; ou adicionar um allowlist de origem (só executar se o bloco vier de `output/<slug>/capitulos/` versionado, não de stdin/bruto).
2. **Máquina de vendas com `POST /api/checkout` e armazenamento local**
- Registro no RTK (l. 313-326 e 320-326): máquina Next.js + FastAPI, rota `POST /api/checkout` com zod, leads vão para `/api/leads/`, backend usa `backend/data/vendas.db`.
 - **Risco**: é uma aplicação web exposta (se deployada). O registro fala em “testar `POST /api/checkout`” e “verificar que o lead chega em `/api/leads/`”, mas não há menção a validação de entrada além de zod, rate limiting, proteção contra spam, CSRF, autenticação de administrador para acessar leads, ou criptografia do banco. Se a máquina for deployada na internet, ela vira um ponto de coleta de contato (e-mail/nome) — a responsabilidade de proteção de dados é do operador, mas a fábrica não documenta expectativas mínimas de segurança para o deploy.
 - **Mitigação recomendada**: adicionar uma seção no AGENTS.md ou no comando `/criar-maquina` com “checklist de segurança de deploy”: autenticação no admin, rate limiting, HTTPS, não logar dados sensíveis, política de retenção dos leads.
3. **Chaves/credenciais nos ambientes**
- O projeto usa várias APIs (OpenAlex, Crossref, arXiv, Semantic Scholar, SciELO, PubMed — V5.4), possível CloudConvert para fallback de PDF, MCP servers.
 - Não há visão de `.env` ou `config/secrets.json` neste analysis (não li tudo). O `requirements.txt` é minimalista. O risco é que credenciais possam estar hardcoded em algum script de pesquisa ou configuração de MCP. Recomenda-se varrer os scripts de `fontes_academicas.py`, `minerar-fontes-academicas.py` e `.mcp.json` para confirmar que não há chaves em texto puro.
4. **Hardlinks e junctions como portabilidade**
- O AGENTS.md descreve hardlinks (`AGENTS.md` → `CLAUDE.md`) e junctions (`.opencode/*` → `.claude/*`). Isso é ótimo para portabilidade multi-IDE, mas cria uma dependência da estrutura de arquivos: se alguém clonar e não rodar `scripts/setup-links.ps1`, vários agentes e comandos não carregam. O risco não é de

segurança, mas de **integridade operacional**: um repositório clonal pode parecer funcional mas ter agentes silenciosamente vazios.

5. Execução de comandos com paths absolutos do Windows

- `compilar-para-pdf.py:97-98` e `subprocess.run([PANDOC, ...])`: se o path do Pandoc/Typst mudar ou se o script for executado por um usuário diferente, falha silenciosamente com `[ERR0] Pandoc não encontrado`. Não é uma vulnerabilidade, mas é uma **falha de confiabilidade** que pode levar o operador a pensar que a compilação funcionou (se o script for mal interpretado).

3.2 Mitigado (bom trabalho documentado)

- `validar-codigo.py` trata `FileNotFoundError` (ferramenta ausente), `TimeoutExpired` e `Exception` em `_rodar` — não propaga falhas brutas.
- Sandbox leve com `tempfile.TemporaryDirectory` + env minimalista para execução de blocos.
- Smoke test dos cards do playbook (`validar-playbook`) com gate executável — o gate vira comando executado, o que valida que o passo é realmente aplicável.
- `auditar-obra.py --estrito` encadeia gates de conteúdo offline (`--sem-rede`) — referências rodam offline e são cacheadas, o que reduz dependência de rede no momento da auditoria.
- R16 (nunca commitar vermelho) + hook pre-commit bloqueia commit se pytest falhar — impede que código com testes quebrados entre no repositório.
- Validação de URLs/DOI reais (`validar-referencias.py`) com reprovação de 4xx/DNS — evita que referências inventadas ou quebradas vão para o livro final.

4. ANÁLISE DE UI/UX

Aqui “UI/UX” é interpretado no contexto do sistema: interação do operador com a fábrica (comandos, relatórios, saídas geradas, máquina de vendas, campanhas).

4.1 Interação operador ↔ fábrica

Bom: - `/esbocar <tema>` como **único ponto de interação humana** (Fase 0) é decisão de UX excelente — depois a fábrica roda autonomamente, o que reduz fricção. Perguntas condicionais (Q5 só se Livro, Q6 só se artigos = Sim) mostram cuidado com o fluxo. - Saída objetiva do `/esbocar` com linha única de próximo passo (`/produzir-obra-completa <slug>` ou comandos individuais) — evita que o opera-

dor fique na dúvida. - Opção de comandos individuais (`/criar-livro`, `/criar-tcc`, `/criar-artigo`, `/criar-ebook`) + full (`/produzir-obra-completa`) dá ao operador controle granular ou atalho total. - Regra 17 (CAMPANHA e MÁQUINA são opcionais, gate `grep 'Autor Digital|centenas de pessoas'` vazio) — a personalização não é opcional apenas no papel; há um gate real que reprova máquinas não personalizadas.

Atenção: - **V5.2 exige relatório MD+PDF por sessão** em `relatorios/`. Isso é ótimo para rastreabilidade, mas se a fábrica roda muitas sessões rápidas, o operador pode acumular muitos PDFs pequenos. Não há menção de retenção/arquivamento — pode virar ruído. - Quando algo falha (ex.: Pandoc ausente, mermaid-cli ausente, timeout), o sistema reporta, mas o **operador precisa interpretar** o relatório. Para um operador leigo, a diferença entre “Markdown expedido, pendência de PDF” e “falha de compilação” pode não ser óbvia sem ler o relatório.

4.2 UI/UX dos entregáveis gerados

- **Livro (PDF via Pandoc+Typst):** template ABNT (`template.typ`), capa gráfica 2D plano, ficha CIP, sumário com `--toc-depth 3`, numeração de seções. Isso é UX editorial sólida — o leitor final não vê a fábrica, vê um livro ABNT profissional.
- **TCC (`template_tcc.typ`):** folha de aprovação, resumo/abstract, numeração progressiva. Adequado à norma.
- **Artigo (`template_artigo.typ`):** layout compacto. Adequado.
- **E-book (EPUB via Pandoc):** sumário clicável, capa 1:1,6, CTA no final. Adequado ao mercado.
- **Lead magnet (HTML+CSS→Chromium→PDF):** A4 com CTA no rodapé. Boa escolha de motor para peça de marketing.
- **Deck (HTML 16:9 navegável + PDF):** apresentação offline via navegador. Boa.
- **Campanha (artes PNG via Chromium, cronogramas MD+PDF):** as artes são geradas por formato (Instagram, LinkedIn, WhatsApp) com variação por envio — R-CP-6 (unicidade por MD5) garante que não são duplicadas. O cronograma tem 4 dimensões (o quê / por quê / como / quando) — excelente UX de uso.

4.3 Máquina de vendas

- Gerada por `/criar-maquina <slug>` com 1:1 por coleção, snapshot das campanhas, personalização por nicho (8 pontos) com gate real — bom.
- O registro no RTK (l. 313-326) mostra que houve bugs reais no template inicial (campo `produto` default desconectado de `config/produtos.json`, checkout page sem campo de e-mail nomeando → 404, form urlencoded vazio → 500 no `request.json()`). Os bugs foram corrigidos, mas é um sinal de que a **UX do desenvolvedor ao integrar a máquina** (qualidade do template inicial) teve falhas que só apareceram no uso.

4.4 Pontos de fricção de UX

1. **Caminhos longos do Windows** — `output/<raiz>/<código-obra>/<pfx>-<seq>-<nome>/` ainda pode chegar perto do MAX_PATH (260) em hubs com coleções grandes. A regra V5.1 encurtou de ~197 para ~150 chars, mas em cenários de campanhas com 12 materiais e múltiplos volumes, o caminho completo pode voltar a ficar grande. A validação `nomes_curtos.py` existe, mas o operador pode ficar confuso quando um material “não é encontrado” por causa de path truncado no Windows.
 2. **Explicação de “coleção vs. série vs. hub”** — O AGENTS.md tem um glossário (l. 173-177), mas é um conceito abstrato que o operador precisa internalizar para navegar no `output/` e entender comandos como `/colecão`, `colecão.py --sincronizar`. Para um operador novo, a estrutura de pastas pode parecer sobrenatural.
 3. **Relatórios em MD+PDF para cada sessão** — boa prática, mas sem uma visão consolidada (ex.: index-page com todos os relatórios), o operador precisará navegar pela pasta `relatorios/` cronologicamente.
-

5. SÍNTESE

Dimensão	Veredito	Prioridade de atenção
Arquitetura Limpa	Sólida no núcleo (registro declarativo, separação de fases, deterministicismo, resiliência). Degenerescências: paths de binaries hardcoded, autor fixo no código, rastros de gravação fora do hub, necessidade de guardião de caminho.	Média
Código Limpo	Legível, docstrings, funções com nome, separação de validação/execução. Repetição de imports opcionais, montagem de frontmatter em string f, heurística de detecção de linguagem sem teste visível.	Baixa
Segurança	Boas práticas de sandbox leve, pre-commit, gates offline, validação de referências. Riscos: execução arbitrária via <code>--executar</code> (baixo para uso solo, subiria com colaboradores/prompts genéricos), máquina de vendas sem checklist de segurança de deploy documentado, possibilidade de credenciais hardcoded em scripts de pesquisa/MCP (não verificado neste analysis).	Média (para o caso de deploy da máquina)
UI/UX	Boa interação operador (Fase 0 interativa + autonomia + opção full/individual + gate de personalização). Entregáveis editoriais com qualidade. Fricções: conceito de hub/coleção/série abstrato, caminhos longos, relatórios por sessão sem index consolidado.	Baixa

6. O QUE NÃO ENTRA NESTE RELATÓRIO (por escopo)

- Análise de qualidade do conteúdo editorial (dossiê, sumários, capítulos) — seria necessário ler obras específicas em `output/`.
- Análise da suíte de testes (`pytest`) com execução real — não rodamos a suíte neste analysis.
- Varredura de `.env`, `.mcp.json`, scripts de `fontes_academicas`, `.claude/settings.json` em busca de credenciais.
- Análise de UI da máquina de vendas em si (Next.js + FastAPI) — preciso ler o template gerado.

Se quiser, posso aprofundar qualquer um dos tópicos acima lendo os arquivos concretos que faltaram (ex.: `tipos_obra.py`, `scripts/hooks/pre-commit`, `templates/`, `skills/`, um livro pronto em `output/`).

